Fully-Resident Weight Storage for Mixed-Precision SLM Inference on a 32-MB PSRAM MCU

Shizhou Xu

Abstract—We show that mixed-precision quantization enables fully-resident weight storage of a dense small language model (SLM) on a 32-MB PSRAM microcontroller. A 63.91M-parameter model quantized to q3e4 (embedding/output q4, attention and FFN q3) occupies 27.3 MiB and runs entirely in on-board PSRAM with a 192-token context window (FP16 K/V cache) and no PC host at inference time. Measured decode throughput is 0.620 tok/s (1.61 s/token) after int16-activation, zero-copy packed GEMV, activation-quantization hoisting, and a 32-bit batch-unpack / four-way-unrolled / -O3 pass, a 3.49× cumulative speedup over the packed float-dequant baseline (2.13× format/activation change, 1.23× hoisting, 1.33× unpack); the GEMV kernel dominates decode time, so kernel-level gains pass through almost unchanged to decode latency. A pre-registered accuracy bound (fixed-text cross-entropy within 1.15× of q4, plus a deterministic 30-prompt well-formedness score $\geq 80\%$) separates the feasible configurations: q2 crosses the accuracy cliff, q4 exceeds the 32-MB budget, and q3e4 is the only configuration that passes both criteria on the host recipe (24/30; the device run scores 22/30); it keeps about four times more PSRAM headroom and is selected for deployment. All arithmetic is documented and reproducible from the open artifact.

Index Terms—Edge AI, on-device inference, large language models, mixed-precision quantization, PSRAM.

I. Introduction

PUTTING an entire dense language model on a microcontroller is a capacity problem before it is a compute problem. On a fixed 32-MB PSRAM budget, a 63.91M-parameter SLM needs about 122 MiB in FP16 and 61 MiB in INT8 (244 MiB in FP32), all far beyond on-board memory; weights must therefore be compressed until they fit, and any configuration that does not fit is simply not a candidate, regardless of its arithmetic quality.

Prior work avoids this wall [11] in two ways. A 180.9M-parameter MoE design (p-for-llm) [1] trades dense residency for sparsity: only the routed experts are active, its ∼44 MiB of state is split between flash XIP and PSRAM, and part of the PSRAM weights are transferred from a host over USB at boot, so the host is absent during inference yet still required at startup; its ∼9 tok/s reflects top-1 routing over 29 experts per layer. The PIE series [2] instead accelerates kernels on models far below the capacity boundary (0.61→4.31 tok/s) and does not address how much dense model can reside. A recent survey of MCU LLM systems [4] finds fully-on-chip evidence only at the 256–412 KB scale and one 5.81 MB ternary ViT [14] with external PSRAM; a fully-resident dense 27.3 MiB model on an ESP32-class device falls in a region that is not covered. For scale, an ESP32-S3 board has been reported at 22.6 tok/s on a TinyStories-scale model [12].

This letter answers two questions within one concrete design point: how far a dense SLM can be compressed and still reside fully in 32 MB of PSRAM, and what latency that residency costs once the storage problem is solved. We quantify the deployment frontier with a pre-registered accuracy bound, measure the decode-time split between GEMV and the rest of the forward pass, and show that the GEMV kernel—not PSRAM bandwidth—dominates latency, which is why kernel-level gains from 2.13× to 3.49× appear almost one-to-one at the system level. We do not claim a new quantization method or a new system architecture: the contribution is an auditable capacity point, a measured latency-cost attribution, and a reproducible artifact.

II. Method

Quantization scheme. Following the group-wise post-training quantization line [6]–[8], q3e4 stores the token embedding and the tied output layer at q4_0 (18 bytes per 32 weights: one fp16 block scale plus 16 packed nibbles) and all attention and FFN linear projections at q3_0 (14 bytes per 32 weights: fp16 scale plus 12 packed 3-bit values). Activations are quantized to int16 per GEMV call [13]; RMSNorm weights and biases remain FP32; the deployed K/V cache is FP16 at 192 tokens (Sec. III). A KMCU image is a single flat file with a 4,480-byte header followed by per-tensor packed blocks; the q3e4 image is 28,632,448 bytes (27.3 MiB).

Accuracy bound (pre-registered). A configuration is deployable only if it simultaneously satisfies (i) per-token cross-entropy on a fixed text within 1.15× of the q4 reference, and (ii) a deterministic well-formedness score of at least 80% on 30 fixed prompts (length, repetition, ending, echo and control-character rules). The thresholds, prompt set and rule checker were fixed before any q3e4 measurement and are archived in the artifact at commit be624d7 (2026-11-05, 3.6 h before the first q3e4 device log); they are not adjusted afterwards. Because the frontier is host-simulated, the score is evaluated with the host recipe, and the device score of the deployed image is reported as a cross-check. The q4 reference cannot run on the device (36.0 MB exceeds the budget), so it is simulated on a host with the identical block-32 recipe.

Bit allocation rationale. The embedding and output matrix (6400×768) is kept at q4 because it carries lexical identity and is reused by the tied head; attention and FFN

Guangxi Medical University, Nanning, China; e-mail: X16717701114@outlook.com. Manuscript received November 6, 2026.

are both q3 in q3e4 (the mixed variant promotes attention to q4). The all-q3 variant is 0.61 MB smaller with nearly identical CE, but it scores 21/30 = 70% on the rule set, below the 80% gate, so keeping the embedding/output at q4 is justified by the rule criterion rather than by CE alone.

Zero-copy packed GEMV. Dequantizing to an expanded copy would multiply the resident footprint (8.5× for FP32, 2.1× for INT8) and add per-element traffic. Instead, the packed image is used in place: each 32-weight block is read from PSRAM, unpacked on the fly into signed integers, and dotted with the int16 activation vector; the fp16 block scale is applied to the integer partial sum. The inner loops unpack eight 3-bit fields from three bytes with constant shifts, keep four independent integer accumulators, and process two output rows per pass; the final build adds -O3. Our implementation builds on the Kestrel-MCU engine [3]: the q4_0/q3_0 formats and packed GEMV are reused, while the int16 integer path, 3-bit unpack, quantization hoisting, block-sum correction, two-row unrolling and FP16 K/V cache are new. Kestrel's optional PIE w8a16 path expands weights to int8 (61 MB here), which cannot fit the 32-MB budget; packed 3-bit storage is the binding constraint.

Context and KV budget. The deployed context is 192 tokens with FP16 K/V caches: $2 \times 8 \times 192 \times 4 \times 96 \times 2$ B = 2,359,296 B, i.e. the same PSRAM as a 96-token FP32 cache (for reference, 96 tokens with FP32 K/V was the earlier configuration; Sec. III reports the measured effect of the FP16 cache).

Model. The 63.91M model (8 layers, hidden 768, 8 query / 4 KV heads of 96, FFN 2432, vocab 6400 tied) is trained from scratch on a local 12-GB GPU: pre-training (39,695 steps, final loss 2.01) and supervised fine-tuning (56,608 steps, final loss 1.71), then quantized to q3e4. The architecture follows Qwen3-style QK-norm [5] and the MiniMind recipe [10]; a QK-norm correction in the deployed kernel is an enabling fix, not a contribution. Integration. The target board is an ALINTEK ESP32-P4 (360 MHz, 16 MB flash, on-board 32 MB PSRAM) [9]; the 28.63 MB image and tokenizer live on a microSD card and are loaded once at boot. The board also acts as a USB host for a HachimoDock pet [15] through a CH343 bridge (4 Mbaud); all inference runs on the board and no PC or network participates after flashing.

III. Results

Memory budget (Table I). All weights and caches are allocated once at boot and their sizes are auditable: the q3e4 image (28,632,448 B), the KV cache (2,359,296 B), 209,860 B of runtime PSRAM allocations (heap, USB-host descriptors and protocol buffers) and small internal-RAM workspaces leave 2,352,828 B of PSRAM free after KV initialization, measured at runtime.

Decode latency and GEMV share (Table II, Fig. 1b). The baseline is the earlier packed path with per-weight floating-point dequantization (an expanded copy cannot fit the budget); replacing it with int16 activations and integer packed weights reduces decode from 5,622 to 2,639 ms/token (0.379 tok/s, 2.13×). Hoisting the per-token activation quantization out of the 6,400-row output-head loop and replacing per-weight offsets by block-sum corrections reaches 2,152 ms/token (0.465 tok/s), and the 32-bit batch-unpack / four-way / -O3 pass reaches 1,612 ms/token (0.620 tok/s), i.e. 3.49× cumulative. Cycle counters give GEMV shares $r = 0.988$, 0.985 and 0.977 on the three builds. Amdahl's law with the post-change r and the stage gains 2.13×, 1.23× and 1.33× gives 2.10×, 1.22× and 1.32×; using the pre-change baseline GEMV share ($f = 0.994$) for the first stage gives 2.13×, matching the observed 2.13×. With $r \approx 0.98$ this near-identity is expected: GEMV dominates decode, so the comparison is an attribution rather than an independent speedup prediction. Counting one full read of packed weights (28.63 MB) plus one complete KV pass (2.36 MB) gives 30.99 MB/token, i.e. 11.7, 14.4 and 19.2 MB/s for the three builds (weight-only 10.98, 13.5 and 18.2 MB/s), far below the 95.9 MB/s sequential-read microbenchmark (1 MB buffer, four accumulators). The scalar unpack/dot instruction stream, not memory bandwidth, is therefore the limiting factor.

FP16 K/V cache. Storing the K/V cache in FP16 doubles the context from 96 to 192 tokens at the same 2,359,296 B. On the same prefix, window-0 CE changes from 6.662094 to 6.660668 ($\Delta = 0.0014$, 0.02%) and decode time stays within run-to-run variation (measured on the pre-optimization kernel), so the larger context is adopted at no measurable accuracy cost. The context change itself has a large modeling effect: one 192-token window scores CE 5.970 versus 7.014 for resetting every 96 tokens (PPL 392 vs. 1,112), i.e. about a 15% reduction in CE.

Accuracy frontier (Table III, Fig. 1a). Host simulation with the identical windowed recipe (each 96-token window

TABLE I
Deployed memory budget on a 32-MB PSRAM ESP32-P4 (measured). Internal-RAM rows are outside the PSRAM total.

Item	Bytes
q3e4 weights	28,632,448 (27.3 MiB)
KV (192 tok, FP16)	2,359,296
Runtime PSRAM (heap/USB)	209,860
Norm arena (int. RAM)	58,368
Scratch+act (int. RAM)	52,608
PSRAM free after KV	2,352,828
PSRAM total	$\leq$ 33,554,432

TABLE II
Decode-time ablation (device; the first row is the earlier packed float-dequant path, not an expanded copy).

Config	ms/tok	Speedup	r	MB/s
Packed, per-weight float	5,622	1×	–	–
int16 + packed	2,639	2.13×	0.988	10.98
+ hoisted quant.	2,152	2.61×	0.985	13.5
+ batch unroll, -O3	1,612	3.49×	0.977	18.2

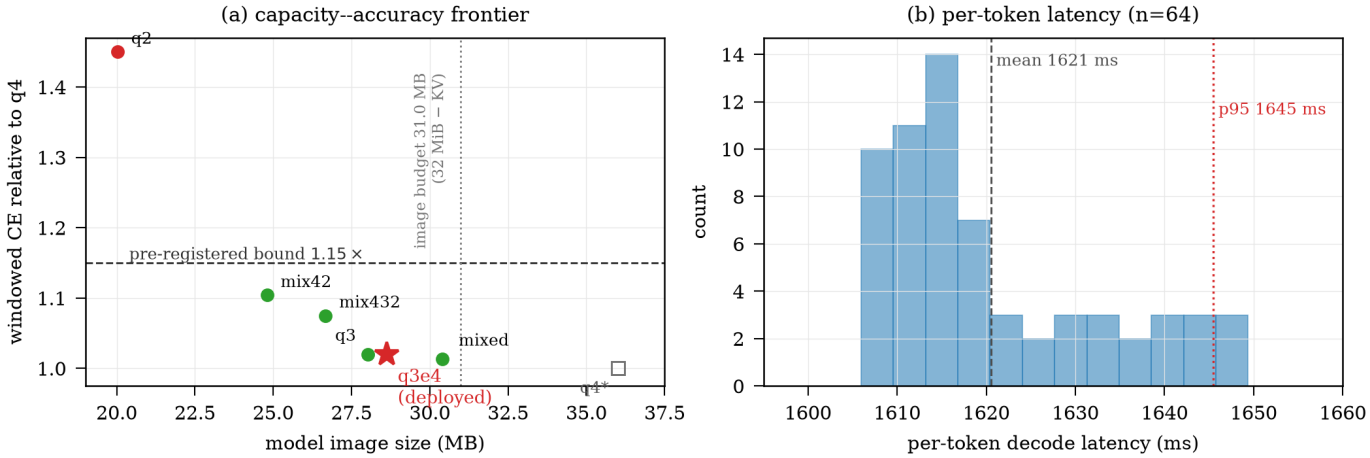


Fig. 1. (a) Capacity–accuracy frontier: windowed CE relative to q4 versus model image size. The 1.15× line is the CE criterion; the dotted line is the image budget left after the 192-token KV cache and runtime allocations (31.0 MB). q3e4 is the only configuration that also passes the rule criterion on the host recipe. q4* is the CE reference (36.0 MB, does not fit). (b) Distribution of per-token decode latency over 64 measured steps (mean 1,620.5 ms, p95 1,645.5 ms).

TABLE III
Capacity–accuracy frontier (32-MB PSRAM). Sizes, CE and rule scores are host-simulated with the identical windowed recipe; the last column is device decode time on the final build.

Config	Size (B)	CE	CE/q4	Rule	ms
q2	20,030,848	10.074	1.450	0%	1,352
q3	28,018,048	7.090	1.020	70%	–
q3e4	28,632,448	7.087	1.020	80%	1,612
mixed	30,401,920	7.037	1.013	73.3%	2,017
q4	36,005,248	6.949	1.000	73.3%	n/a
fp32	–	6.973	1.003	80%	n/a

TABLE IV
Constraint comparison. Prior-work throughputs are as reported by their authors; this work is measured on the final build.

Work	Model	Residency / boot	tok/s
p-for-llm	180.9M MoE	partial; USB boot	∼9
PIE	–	kernel accel.; no capacity	0.61–4.31
This work	63.91M dense	full 27.3 MiB; no PC host	0.620

scored independently) places q4 at CE = 6.949 (PPL 1,042) and q3e4 at 7.087 (1.020×, PPL 1,197); q2 reaches 10.074 (1.450×, PPL 23,710). On the English corpus (Apache-2.0 license text, 288 tokens) the ordering is unchanged: q3e4 5.676 (1.048×) versus q2 9.905 (1.829×). The device reproduces q3e4 window-by-window with a host/device CE difference below 0.001, which validates the recipe for the configurations that run on both sides; the q4 reference cannot run on the device and relies on the same host recipe. The 30-prompt well-formedness score is 24/30 (80%) for q3e4 on the host recipe and 22/30 (73.3%) on the device run, 24/30 for fp32 and 22/30 for q4; q3 scores 21/30 and q2 0/30. Only q3e4 passes both criteria on the host recipe; the mixed configuration (30.40 MB) meets the CE criterion (7.037, 1.013×) but fails the rule gate (22/30) and leaves only 0.58 MB of PSRAM, whereas q3e4 leaves 2.35 MB. We therefore report a capacity–accuracy frontier whose deployable point is q3e4; mixed and the smaller variants are CE-feasible but rule-infeasible.

Two further configurations with larger headroom meet the CE criterion but fail the rule gate: mix42 (24.80 MB; embedding/attention q4, all FFN q2) reaches CE 7.673 (1.104×) and 60.0% (18/30), and mix432 (26.67 MB; FFN down q3) reaches 7.467 (1.075×) and 66.7% (20/30), so the CE frontier below q3e4 is graded while the rule gate is not.

Positioning (Table IV). The closest points are a sparsity-assisted MoE with partial residency and a boot-time host transfer, and vector-kernel acceleration on models well below the capacity boundary. We do not compare throughput directly: the table positions constraint differences, and the reported throughputs are cited as reported by their authors.

Decode and startup. Weight loading from a TF card takes about 20.6 s, dominated by the card read (the packed image needs no post-load unpacking). A 32-token run gives mean 1,629 ms and p95 1,648 ms, and a 64-step measurement gives mean 1,620.5 ms, median 1,616.0 ms, p95 1,645.5 ms (std. 12.4 ms), i.e. a narrow distribution without long tails; latency rises by about 2.7% from position 0 to 31 as attention reads a longer K/V cache. Prompt prefill is processed token-by-token (no batched prefill), so a short query plus a 32-token reply takes on the order of one to two minutes. The mixed configuration (30.40 MB) also boots and generates in the full standalone firmware, confirming that it is deployable; the other frontier variants were evaluated on the host only.

IV. Discussion and Limitations

The measured split changes the design story: the deployment is GEMV-bound ($r = 0.977$ in the final build), and the unpack/dot instruction stream is the throughput limiter, not the PSRAM bandwidth. The practical

consequence is that kernel work survives at the system level (3.49× cumulative); the final build costs about 9.1 cycles per weight (360 MHz × 1.612 s / 63.9M), consistent with an unpack-and-integer-dot instruction stream, and wider unpacking has already been applied, so further gains would require a different core/ISA or custom instructions rather than fewer bytes read. The 192-token context is obtained at no extra memory by storing the K/V cache in FP16 (2.25 MiB for 192 tokens); measured windowed CE changes by 0.0014 and decode time stays within run-to-run variation on the pre-optimization kernel, so the FP16 cache is adopted for deployment while FP32 K/V at 96 tokens remains a conservative fallback. The model itself is deliberately small and trained on a single local GPU; its absolute quality is not the claim—accuracy measurements only delimit the feasible set. We report no energy-per-token figure (no power meter was available) and no standard benchmark score; a fixed held-out text and a deterministic rule set keep the comparison auditable but bound external validity. The usage model is single-turn: prompt prefill is token-by-token and replies are short, so the reported throughput is not a conversational latency. The artifact, training logs, evaluation scripts and the self-trained q3e4 image are public for reproduction.

Reproducibility and artifact. The public repository https://github.com/KAYOYAK/hachimodock-minimind-esp32p4 (release v0.2.1) contains the self-trained q3e4 image (SHA-256 feb574f966980a71c53c6b1f56daf054f267ed425b72ee5ce93f6002ccf87f0), the firmware sources, the 30 prompts with the rule checker, the windowed-CE scripts, the held-out text and token ids, and the six alternative quantized images used for the frontier. Every number in Tables I–III and the this-work row of Table IV can be regenerated from these files; the host simulation uses the identical block-32 recipe, and the host/device agreement above bounds its error for the configurations that run on both sides.

V. Conclusion

A 63.91M dense SLM can reside fully in 32 MB of PSRAM at 27.3 MiB with a 192-token context and no PC host, and decode latency is dominated by the packed GEMV kernel rather than by memory bandwidth, so kernel-level work transfers almost directly to decode speed. The 0.620 tok/s figure is decode-only: boot takes about 20.6 s, prefill is token-by-token, and the device is used for short single-turn queries. The capacity–accuracy frontier, with q2 beyond the accuracy cliff and q4 beyond the budget, is measured under a pre-registered bound and is reproducible from the public artifact.

Acknowledgment

The author used the DeepSeek Harness AI coding agent (DeepSeek model) to assist with the ESP32-P4 firmware implementation (model loading, packed-weight GEMV kernels, USB host protocol bridge), quantization and measurement tooling (Sections II–III), experiment automation, and manuscript editing. The AI system is not an author. The author independently verified all generated code, measurements and text and takes full responsibility for the accuracy, originality and integrity of this work. This work received no external funding, and the author declares no conflict of interest.

References

[1] cyfrit, “PFor: Offline 180.9M-parameter LLM and agent inference on ESP32-P4,” GitHub repository, 2026. [Online]. Available: https://github.com/cyfrit/p-for-llm

[2] “From 0.61 to 4.31 tok/s: A complete record of MCU-side LLM inference level-four optimization,” EEWorld forum, Oct. 2026. [Online]. Available: https://en.eeworld.com.cn/bbs/forum.php?mod=viewthread&tid=1369555

[3] m13253246268-ship-it, “Kestrel-MCU: an Apache-2.0 LLM inference engine for RISC-V MCUs,” GitHub repository, 2026. [Online]. Available: https://github.com/m13253246268-ship-it/kestrel-llm-mcu

[4] M. R. Ramdhani, “Language and transformer models with ESP32-class microcontrollers: A systematic review of hybrid deployment and on-device optimization,” 2026. [Online]. Available: https://journals.unpad.ac.id/ijmt/article/view/72405

[5] Qwen Team, “Qwen3 technical report,” arXiv:2505.09388, 2025.

[6] G. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in Proc. ICLR, 2023.

[7] J. Lin et al., “AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration,” in Proc. MLSys, 2024.

[8] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in Proc. NeurIPS, 2022.

[9] Espressif Systems, “ESP32-P4 technical reference manual,” 2025.

[10] Jingyaogong, “MiniMind: train a small language model from scratch,” GitHub repository, 2025. [Online]. Available: https://github.com/jingyaogong/minimind

[11] W. Wulf and S. McKee, “Hitting the memory wall: Implications of the obvious,” ACM SIGARCH Comput. Archit. News, vol. 23, no. 1, pp. 20–24, 1995.

[12] “LLM on a microcontroller: ESP32-S3 at 22.6 tok/s,” technical report, 2026. [Online]. Available: https://insiderllm.com/pdfs/llm-on-microcontroller-esp32-s3.pdf

[13] B. Jacob et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in Proc. CVPR, 2018.

[14] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei, “The era of 1-bit LLMs: All large language models are in 1.58 bits,” arXiv:2402.17764, 2024.

[15] Y. Wang et al., “HachimoDock: An open-source desktop AI hardware,” GitHub repository, 2026. [Online]. Available: https://github.com/YizhengWw/HachimoDock